the DEAP framework, execute the following command on the terminal: *pip3 install deap*. After installation, you can import DEAP with the line of code *import deap*.

A detailed discussion of genetic algorithms and DEAP would require a full article or even more. Remember that neural networks are also a biologically-inspired technique for AI. Additionally, please note that the term 'soft computing' is often used as an umbrella term to denote several of the techniques discussed in this section, as well as neural networks. However, the use of this term is now decreasing.

A few Python gems for AI

Whenever AI and Python are discussed, the focus is often on NumPy, SciPy, TensorFlow, Keras, PyTorch, scikit-learn, etc. However, discussions should not be limited to just these libraries. In this section, we will explore several Python libraries and packages that may not be included in any so-called 'top-ten list of most useful Python libraries for AI' but are still very powerful and useful.

Our discussion starts with PIL (Python Imaging Library) and its fork, Pillow. These libraries offer general image processing functionality and many useful basic image operations, such as resizing, cropping, rotating, and colour conversion, among others. The following Python script, which uses PIL, converts the image file *tux.png* to grayscale. The Python script is self-explanatory, and the grayscale image that it generates is shown in Figure 2. The original image was taken from the Wikipedia article titled 'Tux (mascot)'.

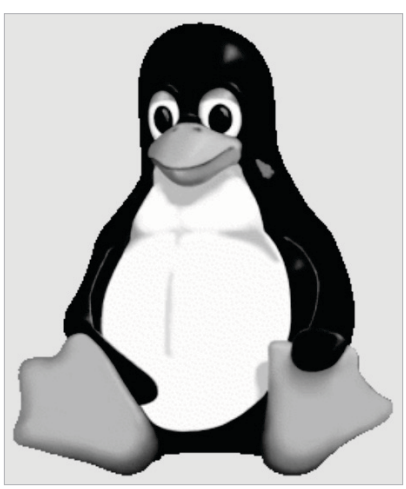

Figure 2: Grayscale image of Tux

```
from PIL import Image
img = Image.open("tux.png")
gray_img = img.convert("L")
gray_img.save("tux_gray.png")
```

If you need to store important data or results for later use, the Python module called pickle can be a useful tool. With pickle, you can convert nearly any Python object into a string representation, which can be easily saved or transmitted. This process is known as pickling. When you need to use the data again, you can reconstruct the original object from the string using unpickling. Pickle is a built-in module in Python, so you can use it without installing any additional libraries. The following Python script provides an example of how to pickle and unpickle a list called *var1*, which contains a mix of integers, strings, and floating-point numbers. The code is self-explanatory and does not require any further explanation. On execution, this Python script will generate a pickle file named 'list1.pickle'. Furthermore, the content of the list will be printed on the terminal.

```
import pickle
var1 = [111, 222, '333', 'AAA',
123.456, 789.123]
with open("list1.pickle", "wb") as file:
    pickle.dump(var1, file)
with open("list1.pickle", "rb") as file:
    var2 = pickle.load(file)
print(var2)
```

Generative AI is a branch of AI that involves creating or generating new content, such as images, music, text, and even videos, using specially designed models. Generative AI is often viewed with a mixture of apprehension and fascination, as exemplified by the deep fake videos that cause fear and the impressive images created by DALL-E 2. Now, we will take a brief look at a Python library called PyOpenGL that can be used for generative AI.

PyOpenGL provides Python bindings to the OpenGL graphics library, which enables developers to create cross-platform graphics applications. It is suitable for various applications, including scientific visualisation, computer-aided design (CAD), and video game development. PyOpenGL can also be employed for generative AI applications that involve graphics processing, such as generating training data for machine learning models or creating generative art. PyOpenGL can be easily installed by using Anaconda Navigator. Now, consider the Python code shown below which uses PyOpenGL (line numbers are included for ease of explanation).

```
1.  from OpenGL.GL import *
2.  from OpenGL.GLU import *
3.  from OpenGL.GLUT import *
4.  import numpy as np
5.  def draw( ):
6.    glBegin(GL_TRIANGLES)
7.    glColor3f(1.0, 0.0, 0.0)
8.    glVertex3f(-0.5, -0.5, 0.0)
9.    glColor3f(0.0, 1.0, 0.0)
10.   glVertex3f(0.5, -0.5, 0.0)
11.   glColor3f(0.0, 0.0, 1.0)
12.   glVertex3f(0.0, 0.5, 0.0)
13.   glEnd( )
14.   glFlush( )
15. glutInit( )
16. glutInitWindowSize(250, 250)
17. glutCreateWindow(b"Generative
    Art")
18. glutDisplayFunc(draw)
19. glutMainLoop( )
```

Let us go through the code line by line. Lines 1 to 4 import the necessary modules from the PyOpenGL and NumPy libraries. Line 5 defines a